INTEGER MULTIPLICATION BELOW $n \log n$ ON MATHLIB'S TURING MACHINES

DAN BROWN* AND KENNY DANIEL

Abstract. We prove in Lean, against Mathlib's `Turing.FinTM2` and `TM2ComputableInTime`, that a fixed finite multi-stack machine multiplies two naturals in worst-case time $O(n(\log n)^{1-\kappa})$ for some $\kappa > 0$. The machine is one host program whose stages are the numeric algorithm of the paper: a CRT reindexing, two linewise resamplings, chirp products, a complex convolution through computed synthetic transforms whose rounds are the simultaneous butterfly layer of §5, pointwise products, and a final rounding. Every stage is a verified machine with an explicit step bound, and the sum of the bounds is $O(n(\log n)^{1-\kappa})$ with $\kappa = \varepsilon c(1-\tau)$ explicit. The vendored library of Kenny Daniel supplies the network, the interchange, the Schönhage–Strassen products and the resampling mathematics. The development has 248 modules and 134 932 lines; every main theorem depends only on `propext`, `Classical.choice` and `Quot.sound`.

1. The theorem

The statement (`Statement.lean`) is Mathlib vocabulary. The input (a, b) is `encodingProd` of two copies of `encodingNatBool`: the LSB-first binary word of a over `Sum.inl` symbols followed by that of b over `Sum.inr`; its length is $n =$ `Nat.size` $a +$ `Nat.size` b. The output is the canonical LSB-first word of ab. The time function of `TM2ComputableInTime` bounds the step count on every input of length n, so the bound is worst case.

```
abbrev pairEnc : ℕ × ℕ → List (Bool ⊕ Bool) :=
  (encodingProd encodingNatBool encodingNatBool).encode

def MulInTime (κ : ℝ) : Prop :=
  ∃ M : TM2ComputableInTime pairEnc encodingNatBool.encode
          (fun p : ℕ × ℕ => p.1 * p.2),
    (fun n : ℕ => (M.time n : ℝ)) =O[atTop]
      fun n : ℕ => (n : ℝ) * Real.log n ^ (1 - κ)

def Statement : Prop := ∃ κ : ℝ, 0 < κ ∧ MulInTime κ
```

Theorem 1 (`Theorem.lean`). *`theorem statement : Statement`.*

The proof is one line: `Fast/Final.fast_statement_of` applied to `Cost/MatchSynLoop.mr_loop_cost_le`. The first is the host program with every bundle instantiated; it takes as its only hypothesis a bound on the cost of the round loops of the synthetic transforms, and the second proves that bound. `FastPathCheck.lean` asserts with `#guard_msgs` that `statement` and the main intermediate theorems depend on `propext`, `Classical.choice` and `Quot.sound` only. The exponent is $\kappa = \varepsilon c(1-\tau) = 296/(10^{11} \cdot 2^{131})$ (`Cost/Total.tot_kappa_eq`).

Date: October 14, 2026.

*Author of *The Da Vinci Code*.

2. The reduction to one program

A *host program* is a procedure table in the structured stack language over the Mathlib input and output stacks and a bank of register stacks. `HostMulWithin body pm v0 time x y` says that from the store holding $x\#y$ on the input register the procedure `pm` halts in the exact final configuration required by `TM2ComputableInTime`, with the product on the output register, in at most $\texttt{time}(n) - 2$ steps.

Theorem 2 (`Fast/Glue.statement_of_dispatch`)**.** *If a host program satisfies* `HostMulWithin` *on every input of encoded length at least n_0 and its time is $O(n(\log n)^{1-\kappa})$ with $\kappa > 0$, then* `Statement` *holds.*

A dispatcher reads up to n_0 symbols into its finite control, answers a short input from a finite table, and otherwise pushes the symbols back and runs the program. `Cost/Total.tot_htime` turns any eventual bound $\texttt{time}(n) \le \texttt{totBound}\, C\, n$ on an $\mathbb{N}$-valued time into the hypothesis `htime` at $\kappa = \texttt{totKappa}$.

3. The numeric algorithm

`Mult/Pipeline.pipelineCoeffs n x y` is the computation of the paper with every ingredient fixed: the parameters $b, p, d, K, T, r, \ell, t_i, s_i, \alpha, \gamma$ of `Mult/Sizes`; the CRT reindexing Φ of `Mult/CRT`; the line maps $\widetilde{A}$ and $\widetilde{B}_0$ with the vendored weight tables; the chirp table; the computed complex convolution through twisted synthetic transforms; pointwise products; the final scaling and rounding.

```
theorem pipeline_correct :
    ∀ᶠ n in Filter.atTop, ∀ x y : ℕ, Nat.size x ≤ n → Nat.size y ≤ n →
      evalBase (2 ^ b n) (pipelineCoeffs n x y) = x * y
```

The moduli are pairwise coprime (`Mult/Moduli.src_good`): group A moduli $Nx + 1$ and group B moduli $Ny + 2$ with N the product of the odd numbers below $4d$, so no prime number theorem is used. The source transform is within $9 \cdot 2^{\gamma} T \log_2 T$ grid units of the exact one (`Mult/SourceErr`), the synthetic convolution error is $\sqrt{2}\, M(3\ell + 1)$ (`Mult/SynthErr`), and the forward and opposite synthetic transforms are exactly ℓ rounds of normalized butterflies on one selected bit per axis followed by per-record twiddles $y^{\mp E}$ (`Mult/Rounds`). $\widetilde{B}_0$ runs α^2 Neumann steps on a window of $\lfloor \sqrt{p} \rfloor / (2\alpha) + 1$ terms (`Mult/FastB0`). Fixed-point products inside the per-entry stages go through the vendored Schönhage–Strassen multiplier.

4. The host program

`Fast/MainProg.mnFast` is the single host program. Its register file (`Fast/MainRegs`) is `MnAll` = `MnR` ⊕ `Fin 64` ⊕ `LReg SXReg`: the stage registers, the 64 registers of the Schönhage–Strassen multiplier, and a reserved copy of the register file of the layer machine for the synthetic transforms and the parameter distribution. The stages, with the cost term of each in `Cost/Total.totBound`:

	stage	semantics	cost term
F1	read x, y	`readInput_exec`	n
F2	parameters, tables	`pmParams_exec`, table machines	$n + B^C, Tp$
F3	scaled digit records	`digitRecords_exec`	n
F4	Φ: stream to padded box	`crtForward_runs`	$Tp\, d^2(1 + \ell^\tau)$
F5, F6	F' on x and on y	`srcP_runs`	resampling, convolution, $TM(p)$
F7	pointwise products	`cst_point_runs`	$TM(p)$
F8	$G' = \overline{F'(\cdot)}$	`srcOpp_runs`	as F5
F9	Φ^{-1}: box to stream	`crtInverse_runs`	$Tp\, d^2(1 + \ell^\tau)$
F10	rounding	`roundStage_exec`	$SM(p)$
F11	carries, output	`carryOut_exec`	n

Here $B = b(n) = \lceil \log_2 n \rceil$, $M(p) = p^{1+\delta}$ with $\delta = 1/8$ is the cost of one p-bit product, the resampling term is $d\,T\,p^{3/2+\delta}\alpha$, and the convolution term is $Tp\,(dK + pK^{\tau-1} + \ell d^{\lambda'} + \log(rp)\log\log(rp))$. The program ends by emptying every register and resetting its local state.

Two parts of `mnFast` are abstract parameters with contracts: the parameter distribution (`MnDistSpec`) and the synthetic transforms (`SynFwdSpec`, `SynOppSpec`, `Fast/ConvSpec`). `Fast/MainCorrect` proves correctness beyond the threshold n_0 and `Fast/MainTime` the bound $\texttt{mnTime} \le \texttt{totBound}\,C$.

Theorem 3 (`Fast/MainStatement.statement_of_specs`)**.** *If for some coefficient width w there are a layout bundle L : `MnLay` w and a distribution bundle `MnDist mnKa mnLb w (mnσL L)`, then `Statement` holds.*

`Fast/BundleMnLay.blLay` builds `MnLay` from the verified layout programs `slyFwd`, `slyOpp` (`Fast/SynLayFwd`, `Fast/SynLayOpp`), the parameter wires v_0 and the transform cost bound. `Fast/BundleDist.bdDist` builds `MnDist` from the verified distribution machine `dsMain`, the rounding parameters (`Fast/BundleRnd`) and a wires program that writes the transform parameters on the reserved layer registers; the wires program is `Fast/DistSynMain.dsyProg` (`Fast/BundleWires`). `Fast/MainFinal.statement_of_wires` needs only the layout bundle and the wires program, and `Fast/Final.fast_statement_of` supplies both from the loop-cost bound.

5. The stage machines

Every machine comes with an `_exec` or `_runs` theorem giving its exact final register contents and an explicit step bound.

- *Input and output* (`Fast/Input`, `Fast/Output`): the operands LSB first, the unary input length, the digit records digit$\cdot 2^{p-b-2}$ zero padded to S; carry propagation in radix 2^b and the Mathlib-encoded output.
- *Parameters* (`Fast/Param*`): a binary table machine with 40 variables of $4\,\texttt{size}\,n + 16$ bits computes every parameter of `Mult/Sizes`, `Mult/Moduli`, `Mult/Pipeline` and `Mult/Chirp` exactly from the unary input length, emits the axis lengths as word streams, and pushes unary copies (`pmParams_exec`, cost $16n + O((\texttt{size}\,n)^4)$ plus the unary copies).
- *Tables* (`Fast/Table*`): the same register machine (`Fast/TableVM`, at most $32(W+4)^2 + 8w$ host steps per atom) computes Machin's π (`tblPi_eq_piQ`), the Taylor exponentials and the weight tables of $\widetilde{A}$ and $\widetilde{B}_0$, the chirp axis tables and the twist tables, as streams of Gaussian numerators in the order the pipeline consumes them.
- *Distribution* (`Fast/Dist*`): one fixed program after the parameter machine writes the record and stream formats read by every stage loader (the per-axis records of the resamplings, the chirp blocks, the layout parameters, the ring header, the layer constants) and the register file between runs of the source transform (`dsAll_exec`, `dsAll_base`; cost $2 \cdot 10^{10}(n + (b+1)^5)$).
- *CRT reindexing* (`Fast/CRT*`): per target axis a walk moving the axis below the preceding ones and a rotation pass; forward with a padding scan, inverse with a deletion scan (`crtForward_runs`, `crtInverse_runs`, round trip `crtRoundTrip_runs`).
- *Resampling* (`Fast/Prep*`, `Fast/Resamp*`): the line machines of $\widetilde{A}$ and of the fast $\widetilde{B}_0$ applied along every axis of the padded box in the order of `tensorZR`, the last axis directly and every other axis through the vendored interchange with the last one (`rsaStage_exec`, `rsbStage_exec`; `rsa_all_lineA`, `rsb_all_tensor`).
- *Chirp and pointwise* (`Fast/Chirp*`, `Fast/Point*`, `Fast/CMul*`): the chirp kernel box from the axis tables by repetition passes and table products (`cst_chirpBox_kernel`); entrywise chirp products, conjugation, the exact scale by 2^γ and the pointwise product (`cst_point_runs`), each through the complex multiplier `cmulStream` over Schönhage–Strassen.

- *Convolution* (`Fast/Conv*`, `Fast/Ring*`, `Fast/Load*`): the twist, the synthetic convolution and the untwist on box streams (`cvBox_convBox`, the instance `Pipeline.convBox n`); the pointwise negacyclic ring products of two record streams by one placed run of the vendored signed ring product (`rpProg_exec`); the loaders of the chirp kernel, the layout stage and the ring header (`ldChirpStage_exec`, `ldLayGenStage_exec`, `ldRpStage_exec`).
- *Rounding* (`Fast/Round`): coefficient i is round$(2^{2b+4}S^2 \operatorname{re} N_i/2^p)$, computed by a sign mask, one Schönhage–Strassen product and a shift (`roundStage_exec`, `rndSquare_exec`).

6. The synthetic transforms and the layer machine

The layer machine. `Node/*` is the machine of §5. One procedure `layer` on the register file `LReg` (the array `io`, the bank of wire registers with temps, a LIFO parking register for the parent's wires during a child call, gate and shift counters, phase, leaf, basis-change and route registers, unary parameter registers) computes the recursive `layerZ` on Gaussian integers modulo 2^w: at a node the array is split into roles, the input phases are applied, the instruction list of the vendored network runs with `gateProg` for gates and `factorProg` for factors (a basis change by slot XORs, a child call with the other wires parked, the basis change undone), the output phases and the unit i^{27k} are applied, and the roles merge. `Node/LayerProc.layer_exec` is proved by strong induction on the level. `Node/TopProc.topProg` is one fixed program computing `topZ` (phase S, kernels on the top chunks, rows and padding, the base-m pieces through `layer`, phase S, the unit $(1-i)^D 2^{-\lfloor D/2 \rfloor}$, the truncating shift); every size and configuration value is read from registers at run time, the pieces from a record consumed by a loop (`topProg_exec`).

The synthetic transforms. `Fast/Syn*` runs the rounds of `Mult/Rounds` on the layer machine. The records stay in the physical order of the layout $L =$ `layPerm` $es\ \ell\ K$, written by the layout programs. For each round h the program `synRec` computes the configuration and piece records of `topProg` from unary parameters (`Fast/SynRec`). A chunk round selects the bits $\rho + K(c_0 + j)$, $j < D$; the highest

$$q_0 = \min\bigl(46\lfloor \log_m D \rfloor,\ D\bigr)$$

of them are top kernels, the rows are the top $a + Kq_0$ address bits, and the remaining $D' = D - q_0$ bits are selected by one piece per nonzero base-m digit of D' on $R_2 = 2^X NW^{J'}$ rows with $2^{a'} \le R_2 < 2^{a'+1}$: the padding at most doubles the volume. The forward and opposite transforms are one loop over the round counter (`Fast/SynFwd.synFwd_spec_br`, `synOpp_spec_br`), with the twiddles of `Fast/Twiddle*`.

Register embedding. The transforms are proved on the register file of the layer machine and run inside the multiplier's file. `Host/Embed.exec_emb`: an execution of p in n steps on T tapes gives an execution of `embProg e ι p` in exactly n steps on T' tapes along an injective e : `Fin`$T \to$ `Fin`T', from every big store that reads back to the small one, ending in a store that reads back to the small final one and is unchanged off the image of e. `Fast/SynFrame.embSynFwd_spec` restates both contracts in the big file.

7. The cost

The analytic bound. `Cost/Total.totBound C n` is C times the sum of the stage terms of the table above, written in the parameters of `Mult/Sizes`. The exponents in force are the vendored $\tau = 1 - 296/10^{11}$ (interchange) and $\sigma = 1 - 418/10^{12}$ (branching), $\beta = 1/2$, $\lambda = 1 - 3/10^{10}$, $\lambda' = 1 - 1/10^{10}$, $\delta = 1/8$, and $\varepsilon = 2^{-75}$, $c = 2^{-56}$ from the sizes. With $Tp \le 48n$ each term is at most a constant times nB^x; the largest exponent is $1 - \kappa$, from the convolution's $Tp \cdot pK^{\tau-1}$, since $B^{\varepsilon c} \le 4K$. The ring-product term $Tp\ \log(rp) \log\log(rp)$ has exponent $1 - \varepsilon + \varepsilon c$.

```
theorem totBound_isBigO (C : ℕ) :
    ∃ κ : ℝ, 0 < κ ∧
      (fun n => totBound C n) =O[atTop]
        fun n : ℕ => (n : ℝ) * Real.log n ^ (1 - κ)
```

`Cost/Growth` holds the bounds on the parameters in powers of B ($d \le B^{\varepsilon} \le 2d$, $K \le B^{\varepsilon c}$, $24n \le Tp \le 48n$, $d\ell \le 2B$, $\alpha \le 4d^{1/2}B^{1/4}$) and the closure lemmas for `IsBigO` against $n(\log n)^e$, including the Schönhage–Strassen step bound $m(\log m + 1)(\log\log m + 1) \le 625\, m^{9/8}$.

The layer recurrence. `Cost/Branching` computes the factor count F of the vendored network's program as the rank sum of its edges and proves $F/NW < m^{\sigma}$ with $NW = 58645352620000$. `Cost/LayerCost` puts the step bound of `layer` in per-volume form: $G(0) \le 200u(1 + n/(rw))$ and $G(j+1) \le (F/NW)\, G(j) + C_1(1+\epsilon)((Km^j u)^{\tau}+1)$ with $\epsilon \le 1$ at admissible nodes. `Layer/Recurrence.layer_cost_bound` solves the recurrence: under $\max(\tau,\sigma) < \lambda < \lambda'$, $\tau(1 + c/\beta) \le \lambda$ and $\sigma + \beta(1-\sigma) < \lambda'$, the base-$m$ pieces of $D' \le d$ chunks plus $q_0 \le Q(1 + \log_m d)$ kernels cost at most $C\, d^{\lambda'}$ normalized, with one constant for all d, K, cutoffs and piece decompositions. `Cost/Total.tot_lam_conditions` checks the inequalities at the exponents in force.

Matching the machines. `Cost/Match*` bounds every stage machine by its term, eventually, with constants depending only on the fixed data (the register count, the interchange bank size, the alphabet, the vendored constants, the width ratios $w \le c_w p$, $W \le c_W p^2$). Two charges go beyond the intended terms. The resamplings interchange each axis with the last one, $2(d-1)$ interchanges of volume $3Tw$ at $\Theta(dTp\ell^{\tau})$ in all; this exceeds the resampling term and is charged to the CRT term $Tp\, d^2(1+\ell^{\tau})$. The layout writer's table word has $rw + N^2 + N + 4$ bits, about $r \cdot w$, and the writer costs a polynomial in B and r, inside Tp (`ml_poly_r_le_T`). The synthetic transforms are matched in three parts: the layout programs against $Tp(dK + pK^{\tau-1})$ (`Cost/MatchSynLay`), the layer of a chunk round against $Tp\, d^{\lambda'}$ through `layer_cost_bound` (`Cost/MatchSynPcs`, `MatchSynTop`), and the records, rest states, parking and twiddles against Tp per round (`Cost/MatchSynOver`).

Theorem 4 (`Cost/MatchSynLoop.mr_loop_cost_le`)**.** *For every C_s and k there is C such that for all large n and every parking cost* stk $\le C_s(r + (b+1)^k)$*, the forward and opposite round loops of the synthetic transforms at the pipeline parameters cost at most $C \cdot$ `totConv`n steps.*

8. The vendored library

The directory `IntegerMultBounds/` is the library of Kenny Daniel (MIT license). The development uses its network (`Networks/`: the compact complex network with $W = 58645352620000$ wires and label dimension $m = 25^3$, its directions and the branching bound `ComplexRank25.branching_bound`), its interchange of address chunks (`Swap/`, `BinaryInterchangeRun.uniform_bound`, cost $CV\max(1,L)^{\tau}$), its Schönhage–Strassen multiplier and signed ring product (`Schoenhage/`), its resampling mathematics (`Resampling/`: the weight tables, the Neumann series of $\widetilde{B}_0$, the operator-norm bound `opNorm_offDiagCLM_le_rpow`), and `ExactRecovery`. Each vendored program runs inside the host as a foreign call with a `HoareTime` contract on host registers (`Host/Core`, `Host/Placed`).

9. Size and verification

`IntMul.lean` imports 248 modules, 134 932 lines in all: `Fast` (144 modules, 78 548 lines), `Node` (20, 21 268), `Cost` (21, 9 054), `Host` (18, 8 732), `Mult` (16, 8 234), `Layer` (19, 6 442), `Bridge` (7, 2 085), `Machine` (1, 512), and `Statement.lean`, `Theorem.lean`. `lake build` of the library passes. `FastPathCheck.lean` asserts with `#guard_msgs` that `statement` and the main intermediate theorems depend on `propext`, `Classical.choice` and `Quot.sound` only, and `MathlibTM2Check.lean` does the same for the bridge theorems of `Bridge/Main` (`mul_quadratic`, `statement_of_endToEnd`, `tm2_of_runsWithin`, `mulInTime_of_computesWithin`); CI runs both.

Email address: jdb1729@gmail.com

Email address: platypii@gmail.com